

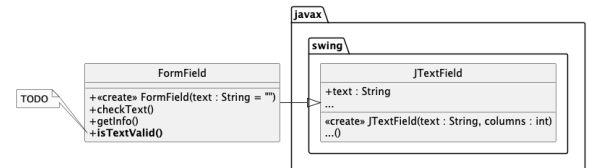
Mise en oeuvre de patrons de conception

Exercice 1

Nous avons une classe `FormField` représentant un champ de texte dans un formulaire. Le problème consiste à valider le contenu du formulaire (i.e., implémenter la méthode `isValid()`). On a plusieurs règles de validation possible. Au choix :

- champ de type email : contient "@" et "."
- champ de type valeur numérique : contient des caractères numériques uniquement
- champ 'nom d'utilisateur' : ne contient que des caractères alpha-numériques
- champ 'mot de passe' : longueur supérieure à 8.

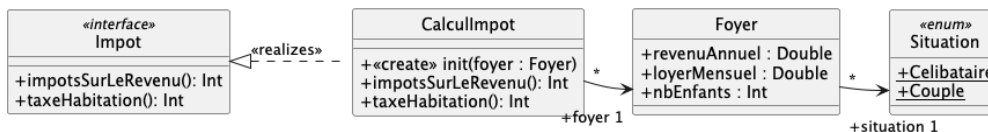
On veut pouvoir facilement ajouter de nouveaux critères.



Exercice 2 : Strategy et/ou Template method

On doit développer une version "simplifiée" d'un système de calcul d'impôts ; pour cela, on prend en entrée, différentes informations : le revenu total annuel du foyer, le loyer mensuel, la situation familiale et le nombre d'enfants du foyer ; avec ces informations, on peut calculer les montants de 1) l'impôt sur le revenu et 2) la taxe d'habitation.

Une première conception UML est la suivante :



Les "règles de calculs" pour les impôts sont les suivantes :

Situation	Revenu imposable = revenuAnnuel - X où X =	Impot sur le revenu	Taxe d'habitation
Couple	20% revenu	revenu imposable x 1/4	deux loyers
Couple avec enfants	20% revenu - 750 par enfants	revenu imposable x 1/(4 + nbEnfants)	1,5 loyers - 50 par enfants
Celibataire	10% revenu	revenu imposable x 1/3	deux loyers + 1000
Celibataire avec enfants	12% revenu - 1'000 par enfants	revenu imposable x 1/(3 + nbEnfants)	1 loyer - 75 par enfants

Toute l'algorithmique est présente dans `impotsSurLeRevenu()` et dans `taxeHabitation()` avec des `if... else...` en cascade pour déterminer tous les cas.

1. Améliorez la conception UML proposée initialement en utilisant le design pattern *Strategy*.
2. Faites évoluer votre conception précédente, pour utiliser le design pattern *Template method*.

Indication : les algorithmes pour `impotsSurLeRevenu()` et dans `taxeHabitation()` ne seront implémentés qu'une seule fois de manière "simplifiée" dans `CalculImpot`.

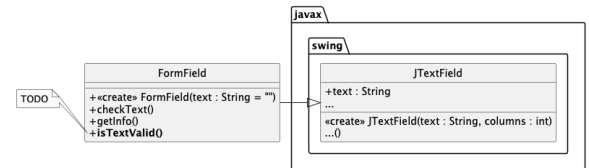
Mise en oeuvre de patrons de conception

Exercice 1

Nous avons une classe `FormField` représentant un champ de texte dans un formulaire. Le problème consiste à valider le contenu du formulaire (i.e., implémenter la méthode `isValid()`). On a plusieurs règles de validation possible. Au choix :

- champ de type email : contient "@" et "."
- champ de type valeur numérique : contient des caractères numériques uniquement
- champ 'nom d'utilisateur' : ne contient que des caractères alpha-numériques
- champ 'mot de passe' : longueur supérieure à 8.

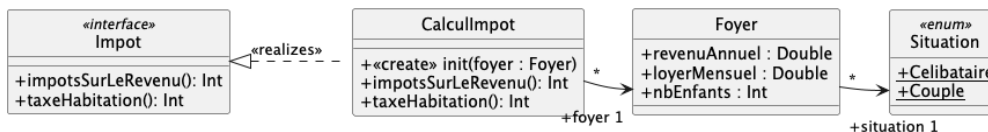
On veut pouvoir facilement ajouter de nouveaux critères.



Exercice 2 : Strategy et/ou Template method

On doit développer une version "simplifiée" d'un système de calcul d'impôts ; pour cela, on prend en entrée, différentes informations : le revenu total annuel du foyer, le loyer mensuel, la situation familiale et le nombre d'enfants du foyer ; avec ces informations, on peut calculer les montants de 1) l'impôt sur le revenu et 2) la taxe d'habitation.

Une première conception UML est la suivante :



Les "règles de calculs" pour les impôts sont les suivantes :

Situation	Revenu imposable = revenuAnnuel - X où X =	Impot sur le revenu	Taxe d'habitation
Couple	20% revenu	revenu imposable x 1/4	deux loyers
Couple avec enfants	20% revenu - 750 par enfants	revenu imposable x 1/(4 + nbEnfants)	1,5 loyers - 50 par enfants
Celibataire	10% revenu	revenu imposable x 1/3	deux loyers + 1000
Celibataire avec enfants	12% revenu - 1'000 par enfants	revenu imposable x 1/(3 + nbEnfants)	1 loyer - 75 par enfants

Toute l'algorithmique est présente dans `impotsSurLeRevenu()` et dans `taxeHabitation()` avec des `if... else...` en cascade pour déterminer tous les cas.

1. Améliorez la conception UML proposée initialement en utilisant le design pattern *Strategy*.
2. Faites évoluer votre conception précédente, pour utiliser le design pattern *Template method*.

Indication : les algorithmes pour `impotsSurLeRevenu()` et dans `taxeHabitation()` ne seront implémentés qu'une seule fois de manière "simplifiée" dans `CalculImpot`.